

Project Journal: Valorant Lite Prototype

Overview

A lightweight, browser-based, single-file First-Person Shooter (FPS) prototype inspired by Valorant. Built entirely using HTML, CSS, vanilla JavaScript, and the Three.js 3D library.

Technology Stack

- **Core:** HTML5, CSS3, JavaScript (ES6+).
- **3D Engine:** Three.js (v0.160.0) imported via CDN.
- **Architecture:** Single-file execution (No build tools, bundlers, or local servers strictly required).

Core Mechanics & Features Implemented

1. Movement & Physics

- **Controls:** Standard WASD movement. Space to jump.
- **Physics Engine:** Custom lightweight physics implementation handling gravity (0.01), friction (0.50), and velocity.
- **Tactical Pacing:** Base movement speed is slowed to simulate tactical strafing (0.2). Holding the knife increases speed by 10% (1.1 multiplier).
- **Collision Detection:** Axis-separated AABB (Axis-Aligned Bounding Box) collision. This allows the player to "slide" smoothly along walls when moving diagonally rather than getting stuck on corners.

2. Weapons & Gunplay

- **Arsenal:**
 - * **Vandal (Primary):** Hitscan weapon. 25 rounds. Headshot: 160 dmg, Bodyshot: 40 dmg.
 - **Tactical Knife (Melee):** Toggle with 'F'. Short range (2.5 units). Headshot: 150 dmg, Bodyshot: 50 dmg. Grants a speed boost.
- **Shooting Mechanics:** Utilizes Three.js `Raycaster` to simulate instant hitscan bullets.
- **ADS (Aim Down Sight):** Holding Right-Click lowers the FOV (from 75 to 50) and centers the weapon model for precision aiming.
- **Visual Feedback:**
 - * Recoil simulation (muzzle climb and gun kickback).

- Bullet tracers (temporary line geometries).
- Bullet holes left on static map geometry.

3. Enemy Entities (Bots)

- **Visuals:** Rendered as 3D stick figures (Cylinders for limbs, Spheres for heads).
- **Health:** 150 HP (simulating heavy armor).
- **Hitboxes:** * Uses a multi-layered hitbox system. An invisible `CapsuleGeometry` covers the torso/legs for consistent body shots.
 - The head is a separate geometric entity flagged specifically to register high-damage headshots.
- **Hitmarkers:** Enemies flash white momentarily when taking damage.
- **Logic:** Currently static/idle targets. They despawn and immediately respawn in a random location upon death.

4. Abilities

- **Smoke (Q):** Throws a projectile in the camera's direction. After traveling a set distance, it expands into a large, semi-transparent purple sphere blocking line of sight. (Cooldown: 300 frames).
- **Dash (E):** Instantly applies a burst of velocity (0.8) in the current movement direction, accompanied by a brief FOV warp effect for "speed" feedback. (Cooldown: 200 frames).

5. User Interface (UI)

- **Start Screen:** Overlays the canvas to capture the mouse via the Pointer Lock API.
- **HUD:** * Center fixed crosshair.
 - Health and Ammo displays.
 - Ability icons with active cooldown states (flashing red when on cooldown, cyan when ready).
- **Killfeed:** Dynamic text popups in the top right corner when an enemy is eliminated.

Technical Analysis & Code Structure

The code is structured linearly for rapid prototyping:

1. **CSS:** Handles all UI overlays, ensuring they sit cleanly on top of the `<canvas>` element generated by Three.js.
2. **State Management (`gameState`):** A single global object tracking player health, ammo, current weapon, and ability cooldowns.

3. **Three.js Initialization:** Sets up the Scene, Camera (YXZ rotation order to prevent gimbal lock), WebGLRenderer, and Lighting.
4. **Map Generation:** Uses a helper function to create boxes and push their bounding boxes to a `colliders` array.
5. **Event Listeners:** Handles Keyboard (WASD, Q, E, R, F) and Mouse (Click to shoot, Right-click to ADS, Mousemove for camera pitch/yaw).
6. **Combat Logic (`shoot()`):** Raycasting logic that differentiates between hitting enemies (checks `isHead` user data), walls, or being out of range (for the knife).
7. **Game Loop (`animate()`):** The core `requestAnimationFrame` loop that calculates physics, updates cooldowns, lerps camera FOV/gun positions (for ADS), and renders the scene.

How to Run Locally in VS Code

1. Open VS Code and create a new folder for your project.
2. Inside that folder, create a new file named `index.html` .
3. Copy the entire HTML/JS code of the game and paste it into `index.html` .
4. **Option A (Easy):** Install the "Live Server" extension in VS Code. Right-click the HTML code and select "Open with Live Server".
5. **Option B:** Simply locate the `index.html` file in your computer's file explorer and double-click it to open it in Chrome, Firefox, or Edge.